

Composition Protocol — Technical Specification

Daml contracts, atomic settlement, per-leg privacy, backend bridge, deployment.

HackCanton Season 3 · MVP as-built + delivery-phase scope · Team: Revotoken Africa

1. System overview

Four architectural layers. Green = real/built, amber = mocked at the API boundary, blue = UI.

Layer	Components	Status
L1 — UI	Proposer view (/proposer), Counterparty view (/counterparty), Observer view (/observer)	Build (React/TS)
L2 — Backend (Node/TS)	Express API :4000 · routes/assets.ts, routes/compositions.ts, routes/audit.ts · ledger.ts · AcceptanceTracker (in-memory)	Real
L3 — Canton Ledger	Participant node · JSON Ledger API v2 :7575 · composition DAR (Daml 3.x)	Real
L4 — Mock boundaries	Price Oracle (mock HTTP :4002) · CIP-0056 registry (mock Token template) · Fiat settlement (console log)	Mocked

2. Daml contracts

Contract	Signatory / Observer	Role
ComposableAsset (interface)	—	Minimal CIP-0056-shaped interface: owner · instrumentId · amount · Transfer choice. Any token implements it (MockToken in MVP).
CompositionProposal	sig: proposer · obs: counterparties	The proposed multi-leg deal. Reject/Expire and Accept choices.
CompositionAgreement	sig: all parties	Co-signed deal. Exposes atomic Settle → maps over legs, exercises Transfer on each, emits SettlementReceipt.
SettlementReceipt	sig: operator · obs: participants	Proof a composition settled. Per-leg scoped: embeds LegSummary; regulator sees receipt, not leg payloads.
AcceptanceTracker	sig: operator · obs: counterparties	Tracks which counterparties have accepted; gates settlement until all required accept.

Composition lifecycle: Propose (create CompositionProposal) → Accept (each counterparty exercises Accept; AcceptanceTracker appended) → when all accepted, Settle (single atomic transaction: map over legs → exercise Transfer on each → all-or-nothing → create SettlementReceipt).

3. Atomicity requirement

R-ATOM-1: Settle MUST execute all legs in one Daml transaction. If any leg's Transfer fails (insufficient balance, unauthorized, conflicting state), the entire transaction reverts — no leg commits. Guaranteed by Daml transaction semantics, not application logic.

R-ATOM-2: No intermediate/half-settled state is ever observable. There is no ordering in which one party holds both legs.

Test: testAtomicSwap — 2 parties, 2 assets, asserts atomic end state; plus a forced-failure case asserting full revert.

4. Privacy requirement (the core claim)

R-PRIV-1: Each leg is visible only to its own stakeholders (leg provider + receiver). A party that is not a stakeholder of a leg **MUST NOT** see that leg's payload — not in ACS, not in the flat stream.

R-PRIV-2: The regulator/observer is an observer of SettlementReceipt only. It sees that a settlement occurred, its timestamp and leg statuses — but is cryptographically excluded from every leg's contents.

R-PRIV-3 (verification): Privacy **MUST** be proven at the ledger level, not asserted in the UI. testAuditorCannotSeeLegs asserts visibleTokens: [] for the regulator while settlementReceipt: found. For the live demo, query the ACS as the observer party and show zero leg contracts.

Per-leg visibility matrix (trade-finance topology):

Leg	Borrower	Lender	Oracle	Regulator
Collateral (Treasury→Lender)	yes	yes	—	—
Cash (USDCx→Borrower)	yes	yes	—	—
Attestation (Oracle→Agreement)	—	yes	yes	—
Receipt (settlement occurred)	—	yes	—	yes

5. Backend → ledger command flow

Route handler → ledger.ts → Canton participant. Three command types over JSON Ledger API v2:

```
create(actAs, templateId, args) → POST /v2/commands/submit-and-wait (CreateCommand)
exercise(actAs, cid, choice, arg) → POST /v2/commands/submit-and-wait (ExerciseCommand)
queryActive(party, templateId) → GET /v2/state/ledger-end → POST /v2/state/active-contracts
```

R-AUTH-1: Every command carries actAs for the required signatories; the Daml engine checks signatory + controller declarations. A backend bug cannot bypass this — the ledger enforces authorization and reverts on failure.

Encoding (verified pattern): Decimal as JSON string; Date as YYYY-MM-DD; Optional as direct value or null; commands use package-id templateIds, ACS queries use package-name (#composition:Module:Template) filters.

6. Ecosystem asset integration

Legs settle real Canton-native assets so the integration is load-bearing, not cosmetic, and each settled leg is a real on-chain test transaction (the activity signal judges count):

- USDCx — cash leg (Canton-native settlement asset).
- CBTC (BitSafe) — Bitcoin-backed collateral leg; also the basis for the BitSafe Decentralization challenge entry.
- cETH (onRails) — CIP-0056-compliant; composes atomically with USDCx in one operation.

7. Deployment topology

LocalNet (development): canton-devkit / LocalNet via Docker Compose — Backend :4000, Next.js :3000, Canton Sandbox :7575, mock services :4001/:4002, optional PostgreSQL PQS.

Shared HackCanton DevNet (submission target): Noders shared node — JSON Ledger API <https://ledger-api-json.participant.hackcanton-01.devnet.naas.noders.services> · OIDC token <https://keycloak.naas.noders.services/realms/noders-appsfactory/protocol/openid-connect/token> · audience <https://hackcanton-01.devnet.naas.noders.services>. Shared node — no sensitive data. Point LEDGER_API_URL at the remote node; no Daml or backend changes from LocalNet.

R-DEPLOY-1: The submitted demo MUST run against the shared DevNet node (or a reproducible LocalNet with documented setup) and be publicly accessible — verified in a private browser window before the Oct 9 deadline.

8. Testing & acceptance

Test	Asserts	Gate
testAtomicSwap	2 parties · 2 assets · atomic end state	Atomicity (R-ATOM-1/2)
testAtomicRevert	one leg fails → whole tx reverts, no partial state	Atomicity
testAuditorCannotSeeLegs	regulator: visibleTokens [] · receipt found	Privacy (R-PRIV-1/2/3)
Ledger API bridge	create/exercise/query round-trip on live node	MVP integration

Definition of done: all four gates green on the shared DevNet node; three role-view UI renders live per-party ACS; observer view shows zero legs while receipt is present; ≥50 compositions settled with zero unexpected failures logged for the Metrics evidence.

9. BitSafe challenge extension (secondary entry)

The Accept step is already multi-party co-signing. For the BitSafe Decentralization challenge, extend to governed settlement:

- R-GOV-1: settlement authorization requires an M-of-N threshold of named operators — a governed action MUST NOT execute below threshold and MUST succeed once met (demonstrated on LocalNet).
- R-GOV-2: name the nodes/operators, state the threshold, and show a failure test (below-threshold rejected, at-threshold accepted). Contribution pool needs only a reproducible LocalNet demo — no Canton node.